

Java vs Kotlin — Manejo de Errores y Excepciones

Comparación rápida para clase

Filosofía general

Java: checked + unchecked, try/catch/finally, try-with-resources

Kotlin: todas unchecked, fomenta Result, sealed, runCatching

try/catch/finally

```
Java:  
try { ... } catch(IOException e) { ... } finally { ... }
```

Kotlin:
`try { ... } catch(e: IOException) { ... } finally { ... }`

Multi-catch y Recursos

Java: `catch(IOException | SQLException e)`
Kotlin: varios `catch`

Java: `try-with-resources`
Kotlin: `use { }`

Manejo funcional (Kotlin)

```
val result = runCatching { risky() }  
result.onSuccess { println(it) }  
    .onFailure { e -> println("Error: $e") }
```

Sealed classes

```
sealed interface FileError {  
    data class NotFound(val path: String): FileError  
    data class Permission(val user: String): FileError  
}
```

Buenas prácticas

Java: `checked` para E/S, `unchecked` para lógica
Kotlin: `Result` o `sealed` para errores de dominio
Ambos: usar recursos automáticos (`try-with-resources` / `use`)